

**CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E
DESENVOLVIMENTO DE SISTEMAS**

Natan de Jesus Soares



PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA:

Atividade: Atividade 02 – Frameworks para Desenvolvimento Destinado para Servidores

**MOGI DAS CRUZES
2026**

Natan de Jesus Soares

Portfólio – RELATÓRIO DE AULA PRÁTICA

Frameworks para Desenvolvimento Destinado para Servidores

Trabalho de portfólio apresentado como requisito parcial para a obtenção
de pontos para a média semestral

Orientador: **VINICIUS MENDES GOMES DA SILVA**

Pós Graduação

Mogi Das Cruzes

2026

SUMÁRIO

1. INTRODUÇÃO E OBJETIVOS
2. ARQUITETURA E PASSO A PASSO DA IMPLEMENTAÇÃO
 - 2.1 Construção da Lógica Principal ([main.py](#))
 - 2.2 Template do Formulário de Entrada ([templates/index.html](#))
 - 2.3 Template de Exibição do Resultado ([templates/resultados.html](#))
 - 2.4 Estilização Visual ([static/style.css](#))
3. PROCEDIMENTOS PRÁTICOS E RESULTADOS OBTIDOS
4. ANÁLISE DE DESAFIOS TÉCNICOS E DIFICULDADES ENCONTRADAS
5. RESPOSTAS AOS QUESTIONAMENTOS DO ROTEIRO
6. CONCLUSÃO
7. REFERÊNCIAS BIBLIOGRÁFICAS

1. INTRODUÇÃO E OBJETIVOS

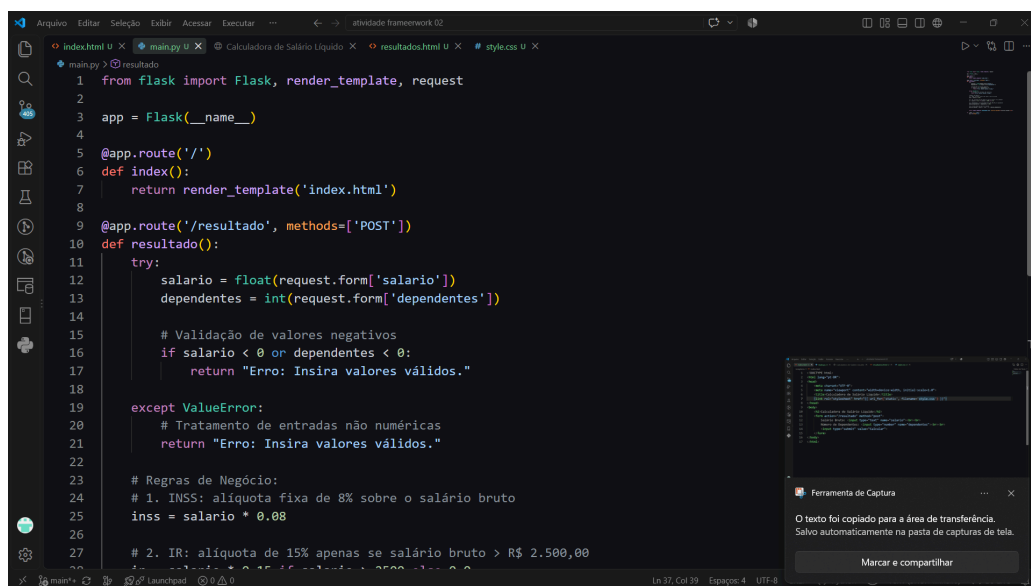
Este relatório apresenta o desenvolvimento de uma aplicação web interativa voltada para o cálculo de salário líquido, construída com o microframework Flask em Python. O objetivo principal da prática foi vivenciar o ciclo completo de construção para servidores, abrangendo o roteamento de páginas, a recepção e o processamento de formulários HTTP via método POST, a organização de visualizadores com a engine de templates Jinja2 e a aplicação de validações no backend para o tratamento de erros e exceções.

2. ARQUITETURA E PASSO A PASSO DA IMPLEMENTAÇÃO

A construção do projeto seguiu rigorosamente a convenção estrutural exigida pelo Flask, separando a lógica de controle, a camada de apresentação e a estilização gráfica em seus devidos diretórios.

2.1 Construção da Lógica Principal (`main.py`)

O desenvolvimento iniciou-se pela criação do arquivo `main.py` na raiz do projeto. Nele, configurou-se o servidor Flask e definiram-se as rotas principais. A rota `/resultado` foi programada para aceitar requisições do tipo POST, convertendo os campos de entrada e aplicando as validações necessárias — interrompendo a execução em caso de entradas não numéricas (`ValueError`) ou valores negativos para salário e dependentes. Na sequência, aplicaram-se as regras de negócio: desconto fixo de 8% de INSS, cobrança de 15% de IR apenas para salários superiores a R\$ 2.500,00 e dedução fixa de R\$ 200,00 por dependente.



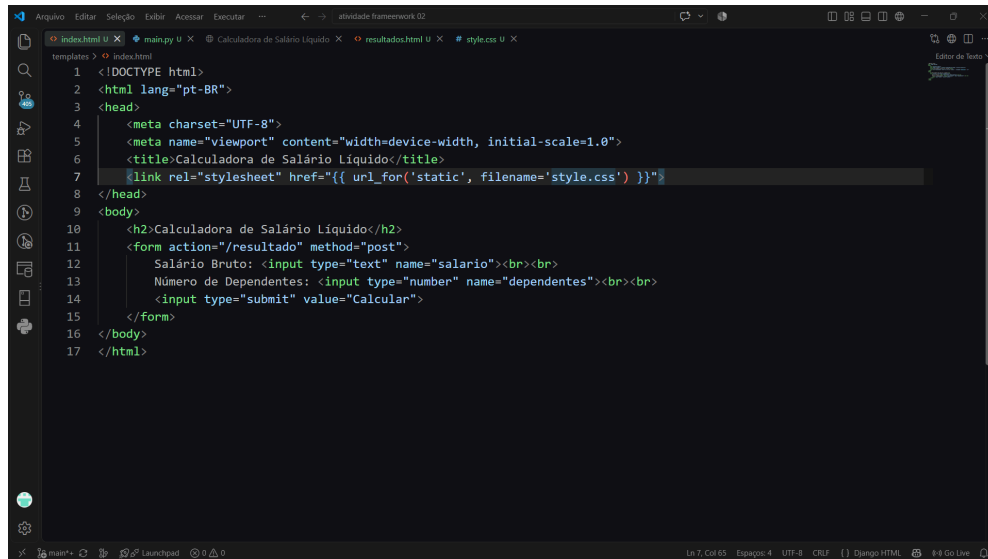
```
1 from flask import Flask, render_template, request
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def index():
7     return render_template('index.html')
8
9 @app.route('/resultado', methods=['POST'])
10 def resultado():
11     try:
12         salario = float(request.form['salario'])
13         dependentes = int(request.form['dependentes'])
14
15         # Validação de valores negativos
16         if salario < 0 or dependentes < 0:
17             return "Erro: Insira valores válidos."
18
19     except ValueError:
20         # Tratamento de entradas não numéricas
21         return "Erro: Insira valores válidos."
22
23     # Regras de Negócio:
24     # 1. INSS: alíquota fixa de 8% sobre o salário bruto
25     inss = salario * 0.08
26
27     # 2. IR: alíquota de 15% apenas se salário bruto > R$ 2.500,00
```

Legenda: Figura 1 – Código fonte do controlador `main.py` com rotas, validações e regras de negócio.

(Arquivo: Captura de tela do `main.py`)

2.2 Template do Formulário de Entrada (templates/index.html)

Na sequência, criou-se a estrutura visual de entrada no arquivo `index.html`, armazenado dentro da pasta `templates/`. O documento traz um formulário direcionado à rota `/resultado` contendo os campos de texto para salário bruto e número de dependentes. Para importar a folha de estilo de forma dinâmica e segura, utilizou-se a função nativa do Flask `url_for('static', filename='style.css')`.

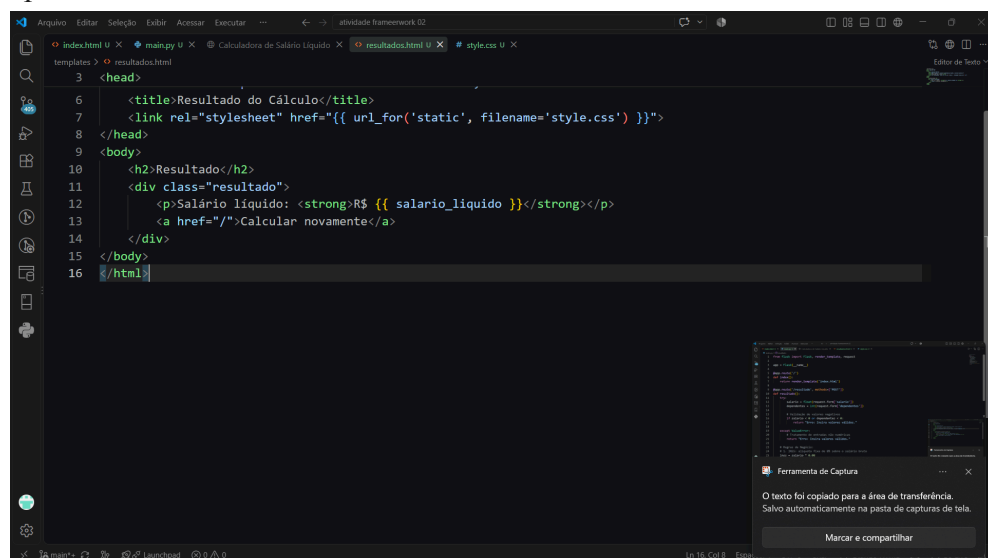
A screenshot of a code editor showing the `index.html` file. The code is written in HTML and includes a form for calculating liquid salary. The form has two input fields: 'Salário Bruto' (Salary Gross) and 'Número de Dependentes' (Number of Dependents), followed by a 'Calcular' (Calculate) button. The code uses Jinja2 templating for dynamic content and Flask's `url_for` function for the stylesheet link.

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Calculadora de Salário Líquido</title>
7   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
8 </head>
9 <body>
10  <h2>Calculadora de Salário Líquido</h2>
11  <form action="/resultado" method="post">
12    Salário Bruto: <input type="text" name="salario"><br>
13    Número de Dependentes: <input type="number" name="dependentes"><br>
14    <input type="submit" value="Calcular">
15  </form>
16 </body>
17 </html>
```

Legenda: Figura 2 – Estrutura do formulário HTML no arquivo `index.html`.
(Arquivo: Captura de tela do `index.html`)

2.3 Template de Exibição do Resultado (templates/resultados.html)

Para apresentar o valor final calculado de maneira limpa ao usuário, desenvolveu-se o template `resultados.html` na pasta `templates/`. O arquivo recebe a variável `salario_liquido` processada no servidor e exibe o valor formatado na tela, disponibilizando também um hiperlink para retornar ao formulário inicial e realizar novas consultas.

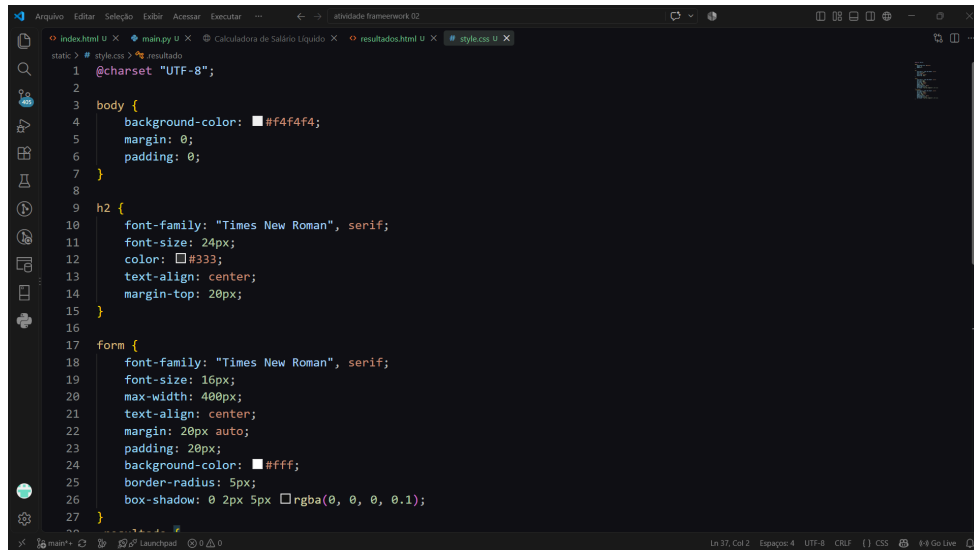
A screenshot of a code editor showing the `resultados.html` file. The code displays the calculated liquid salary using Jinja2 templating. It includes a link to return to the calculation form. A small 'Ferramenta de Captura' (Screenshot Tool) window is visible in the bottom right corner of the editor.

```
3 <head>
6   <title>Resultado do Cálculo</title>
7   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
8 </head>
9 <body>
10  <h2>Resultado</h2>
11  <div class="resultado">
12    <p>Salário líquido: <strong>R$ {{ salario_liquido }}</strong></p>
13    <a href="/">Calcular novamente</a>
14  </div>
15 </body>
16 </html>
```

Legenda: Figura 3 – Código do template `resultados.html` para exibição do salário líquido.
(Arquivo: Captura de tela do `resultados.html`)

2.4 Estilização Visual (static/style.css)

A camada de design foi isolada no arquivo `style.css`, alocado na pasta `static/`. A folha de estilo padronizou a tipografia com a fonte *Times New Roman*, alinhou o formulário ao centro com cartões de fundo branco, cantos arredondados e sombras suaves (`box-shadow`), garantindo uma interface agradável e legível.

A screenshot of a code editor window titled 'atividade framework 02'. The editor shows the contents of a file named 'style.css'. The code defines three CSS selectors: 'body', 'h2', and 'form'. The 'body' selector sets the background color to #F4F4F4, margin to 0, and padding to 0. The 'h2' selector sets the font family to 'Times New Roman', serif; font size to 24px, color to #333, text-align to center, and margin-top to 20px. The 'form' selector sets the font family to 'Times New Roman', serif; font size to 16px, max-width to 400px, text-align to center, margin to 20px auto, padding to 20px, background color to #fff, border-radius to 5px, and box-shadow to 0 2px 5px rgba(0, 0, 0, 0.1). The editor has a dark theme and a sidebar on the left with icons for file explorer, search, and other tools.

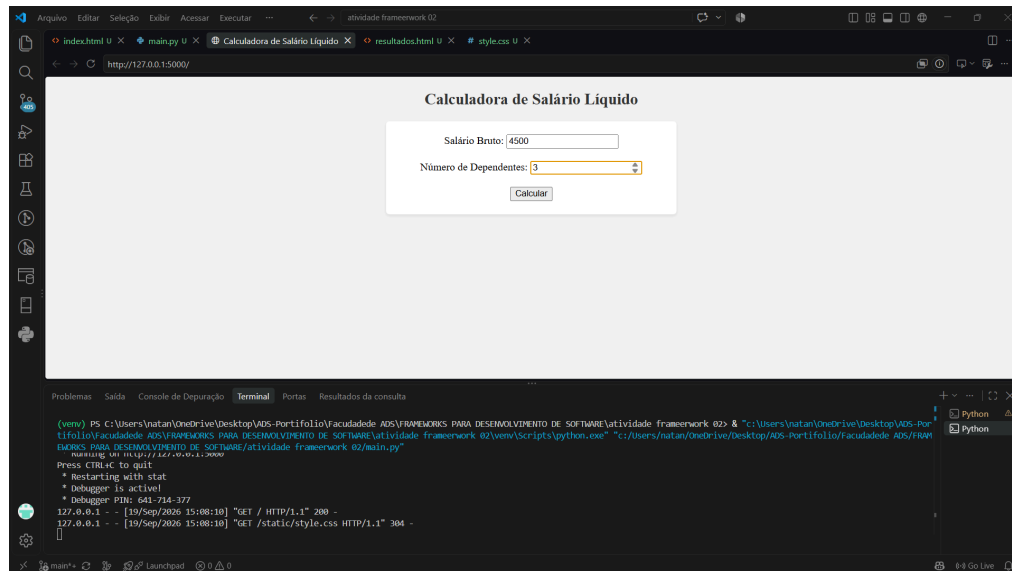
```
1 @charset "UTF-8";
2
3 body {
4     background-color: #F4F4F4;
5     margin: 0;
6     padding: 0;
7 }
8
9 h2 {
10     font-family: "Times New Roman", serif;
11     font-size: 24px;
12     color: #333;
13     text-align: center;
14     margin-top: 20px;
15 }
16
17 form {
18     font-family: "Times New Roman", serif;
19     font-size: 16px;
20     max-width: 400px;
21     text-align: center;
22     margin: 20px auto;
23     padding: 20px;
24     background-color: #fff;
25     border-radius: 5px;
26     box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
27 }
```

Legenda: Figura 4 – Regras de estilização CSS no arquivo `style.css`.

(Arquivo: Captura de tela do `style.css`)

3. PROCEDIMENTOS PRÁTICOS E RESULTADOS OBTIDOS

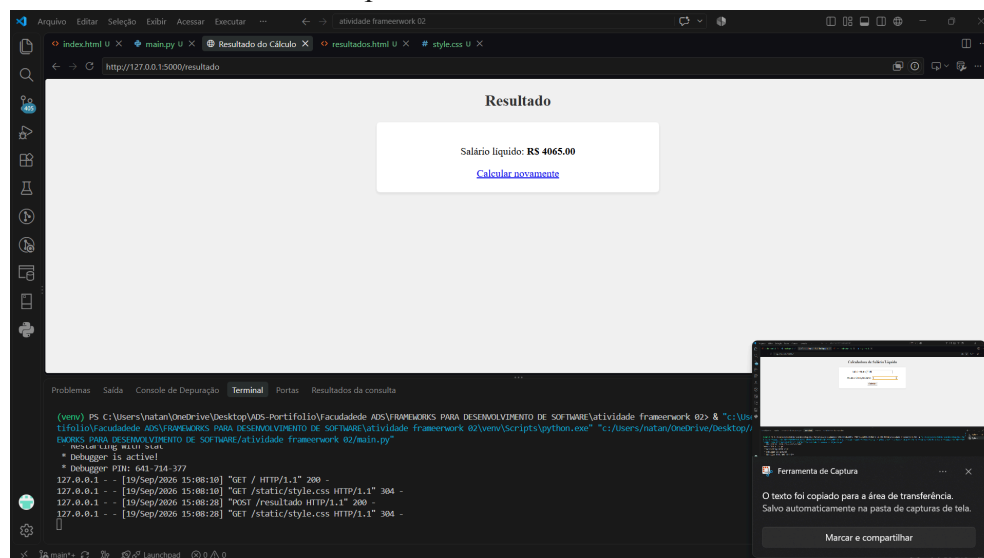
Após a estruturação dos arquivos, o ambiente virtual foi ativado e a aplicação foi iniciada no servidor local (<http://127.0.0.1:5000>). Durante os testes de navegação, preencheu-se o formulário principal com dados válidos — como um salário bruto de R\$ 4.500,00 e 3 dependentes — confirmando que as requisições GET e POST foram processadas corretamente no terminal.



Legenda: Figura 5 – Interface gráfica do formulário em execução no navegador via servidor Flask.

(Arquivo: Captura de tela da página inicial)

Após a submissão dos dados, o servidor efetuou os cálculos (INSS: R\$ 360,00; IR: R\$ 675,00; Adicional por dependentes: R\$ 600,00), resultando no valor exato de **R\$ 4.065,00** exibido na tela de resultados com suporte ao botão de retorno.



Legenda: Figura 6 – Tela de resultado exibindo o salário líquido calculado.

(Arquivo: Captura de tela da página de resultado)

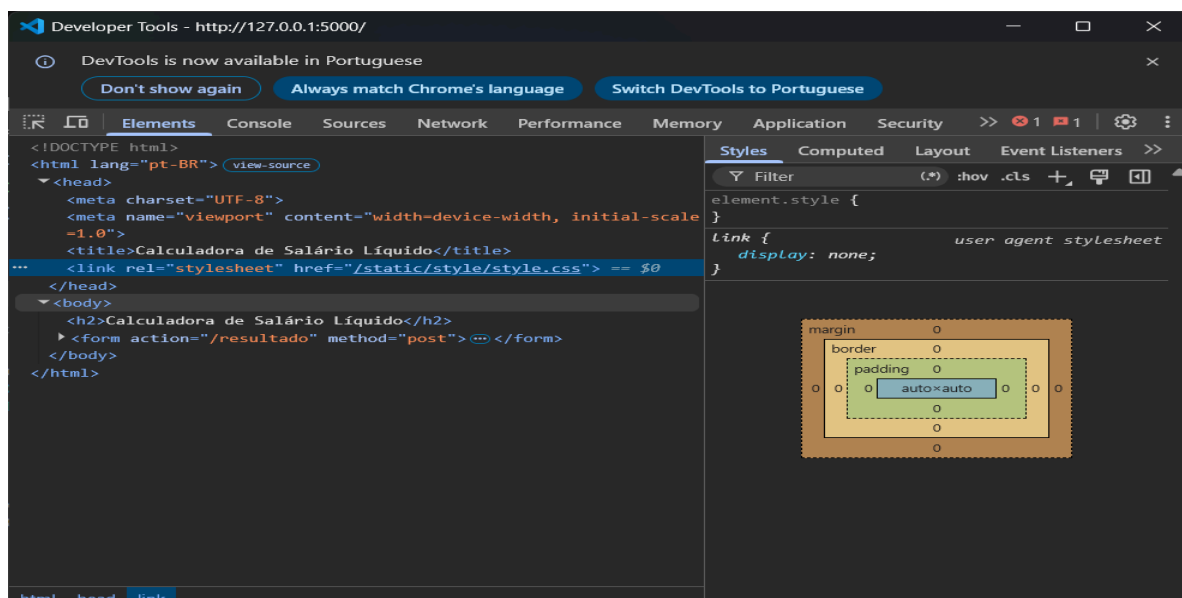
Para validar todas as regras do roteiro, a aplicação foi submetida a seis cenários de teste distintos, obtendo-se os seguintes resultados observados:

Salário Bruto	Dependentes	Resultado Exibido	Memória de Cálculo / Diagnóstico
3000,00	2	Salário líquido: R\$ 2710.00	INSS (240,00) + IR (450,00) - Dep (400,00) = R\$ 2.710,00
2000,00	1	Salário líquido: R\$ 2040.00	INSS (160,00) + IR (0,00) - Dep (200,00) = R\$ 2.040,00
1000,00	0	Salário líquido: R\$ 920.00	INSS (80,00) + IR (0,00) - Dep (0,00) = R\$ 920,00
-500,00	1	Erro: Insira valores válidos.	Interceptado pela validação de salário negativo.
abc	2	Erro: Insira valores válidos.	Capturado pelo bloco try-except (ValueError).
2500	-1	Erro: Insira valores válidos.	Interceptado pela validação de dependentes negativos.

4. ANÁLISE DE DESAFIOS TÉCNICOS E DIFICULDADES ENCONTRADAS

O processo de desenvolvimento envolveu a superação de diversos obstáculos operacionais no ambiente de execução e na configuração do projeto. O primeiro imprevisto ocorreu na tentativa de ativar o ambiente virtual no terminal do Windows, momento em que o PowerShell bloqueou a execução do script `Activate.ps1` retornando o erro de acesso não autorizado (`UnauthorizedAccess`). A falha foi solucionada alterando a política de execução de scripts para o usuário atual por meio do comando `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`. Em seguida, ao tentar listar a estrutura de diretórios, utilizei o comando `dir /s /b` nativo do Prompt de Comando (cmd), que falhou no PowerShell por falta de compatibilidade, sendo substituído com sucesso pelo comando `Get-ChildItem -Recurse -Name`.

Durante a integração dos templates, o servidor disparou a exceção `TemplateNotFound` porque os arquivos HTML estavam salvos na raiz do projeto. O problema exigiu reorganizar a estrutura do projeto para seguir rigorosamente as convenções do Flask, movendo os arquivos HTML para a pasta `templates/` e o arquivo CSS para a pasta `static/`. Com a estrutura ajustada, notei que a página continuava sendo renderizada sem estilos visuais. Ao inspecionar o código fonte através do DevTools do navegador (F12, aba Elements/Network), identifiquei que o HTML estava gerando um caminho incorreto (`/static/style/style.css`), falha corrigida ajustando a tag `<link>` no arquivo HTML para `<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">`.



Legenda: Figura 7 – Inspeção via DevTools no navegador identificando o caminho incorreto do CSS.

(Arquivo: Captura de tela do DevTools)

Além da correção no caminho, foi necessário ajustar a sintaxe do próprio arquivo `style.css`, pois os seletores `h2` e `form` estavam aninhados dentro do bloco `body`, estrutura não suportada em CSS puro sem o uso de pré-processadores. Por fim, ao tentar abrir o arquivo `index.html` diretamente no navegador sem acionar o Python, o sistema retornou o erro `ERR_FILE_NOT_FOUND`, reforçando que templates do Jinja2 necessitam ser processados ativamente por um servidor web. A partir disso, estabeleceu-se a rotina de manter o servidor sempre rodando via `python main.py` e acessar a aplicação estritamente pelo endereço `[http://127.0.0.1:5000](http://127.0.0.1:5000)`, sanando também eventuais falhas de conexão recusada (`ERR_CONNECTION_REFUSED`).

5. RESPOSTAS AOS QUESTIONAMENTOS DO ROTEIRO

1. Com base nos seus conhecimentos, qual a vantagem de validar os dados do formulário no backend? Justifique.

A validação de dados realizada no backend é fundamental para garantir a segurança e a estabilidade da aplicação. Diferente das checagens feitas no frontend — que dependem do navegador e podem ser facilmente ignoradas ou burladas por usuários mal-intencionados —, a verificação no servidor funciona como a camada definitiva de proteção. Ela assegura que apenas dados numéricos e coerentes cheguem às regras de negócio, prevenindo travamentos por exceções não tratadas, corrupção de dados e potenciais vulnerabilidades no sistema.

2. Considerando que o salário líquido depende de várias variáveis (IR, INSS, dependentes), o que seria mais indicado para facilitar futuras atualizações da regra de cálculo? Reflita sobre sua resposta.

Para facilitar futuras manutenções, o mais indicado é adotar o princípio da separação de responsabilidades. Em vez de fixar taxas e valores de dedução diretamente no arquivo de rotas do Flask, deve-se isolar a lógica de cálculo em um módulo de serviço separado e armazenar as alíquotas e faixas salariais em arquivos de configuração ou em um banco de dados. Dessa forma, caso ocorram mudanças na legislação trabalhista, será necessário atualizar apenas os parâmetros configurados, sem a necessidade de reescrever a lógica das rotas ou arriscar a estabilidade da interface do aplicativo.

6. CONCLUSÃO

A realização desta atividade prática permitiu consolidar a aplicação de conceitos essenciais do desenvolvimento web com o framework Flask. A integração do formulário em HTML e da estilização em CSS com as rotas do Python possibilitou criar uma solução funcional, segura e amigável para o usuário. Além disso, a análise detalhada e a resolução dos desafios técnicos enfrentados no ambiente de desenvolvimento reforçaram a importância de seguir as convenções de arquitetura da ferramenta e aplicar boas práticas de programação no servidor.

7. REFERÊNCIAS BIBLIOGRÁFICAS

CURSO EM VÍDEO. **Python 3 - Módulo 1 ao 3**. Produção de Gustavo Guanabara. YouTube, 2018-2026. Disponível em: <https://www.youtube.com/@cursoemvideo>. Acesso em: 18 set. 2026.

FLASK. **Flask Documentation (2.3.x)**. Pallets Projects, 2023. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 18 set. 2026.

HASHTAG PROGRAMAÇÃO. **Criação de Sites e Aplicações Web com Python e Flask**. Produção de Lira Hashtag. YouTube, 2021-2026. Disponível em: <https://www.youtube.com/@HashtagProgramacao>. Acesso em: 18 set. 2026.